



**UNIVERSIDAD  
DE LA FRONTERA**  
FACULTAD DE INGENIERÍA Y CIENCIAS

---

# **Sistema Inteligente de Gestión y Optimización de Rutas de Emergencia**

---

Nombre de la asignatura: Programación Avanzada

Código de la asignatura: ICC708-1

Profesor: Ricardo Gacitua

Integrantes: Diego Aído, Vicente Hernández

Fecha: 10 de julio de 2026



---

## Índice

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| <b>Índice.....</b>                                                            | <b>2</b>  |
| <b>1. Introducción y Contexto del Problema.....</b>                           | <b>2</b>  |
| <b>2. Diseño de Tipos de Datos Abstractos (ADT).....</b>                      | <b>3</b>  |
| 2.1. ADT Incident.....                                                        | 4         |
| 2.2. ADT EmergencyCenter.....                                                 | 5         |
| 2.3. ADT RoadNetwork.....                                                     | 5         |
| <b>3. Estructuras de Datos Propias e Implementación.....</b>                  | <b>6</b>  |
| 3.1. Tabla Hash Personalizada (Separate Chaining).....                        | 6         |
| 3.2. Cola de Prioridad y Max-Heap.....                                        | 7         |
| <b>4. Algoritmos de Ordenamiento (Sorting).....</b>                           | <b>7</b>  |
| 4.1. MergeSort (Ordenamiento por Mezcla).....                                 | 8         |
| 4.2. QuickSort (Ordenamiento Rápido).....                                     | 8         |
| <b>5. Modelamiento del Grafo y Búsquedas de Ruta.....</b>                     | <b>8</b>  |
| <b>6. Escenario Integrado y Flujo Operacional.....</b>                        | <b>9</b>  |
| <b>7. Resultados Experimentales y Análisis de Rendimiento.....</b>            | <b>10</b> |
| 7.1. Análisis de la Tabla Hash.....                                           | 10        |
| 7.2. Rendimiento de la Priority Queue (Max-Heap).....                         | 11        |
| 7.3. Comparación de Algoritmos de Ordenamiento (MergeSort vs. QuickSort)..... | 11        |
| 7.4. Comparación de Algoritmos de Búsqueda de Rutas.....                      | 13        |
| <b>8. Discusión y Conclusiones.....</b>                                       | <b>13</b> |



---

# 1. Introducción y Contexto del Problema

Durante eventos de desastres naturales o situaciones críticas de seguridad pública, las redes de respuesta civil (brigadas médicas, bomberos, vehículos de rescate) se enfrentan a un volumen inmenso de solicitudes concurrentes de auxilio. La toma de decisiones ineficiente, debida a la falta de priorización científica o la incapacidad de calcular trayectos óptimos sobre vías interrumpidas, impacta de forma directa sobre la tasa de supervivencia de las víctimas.

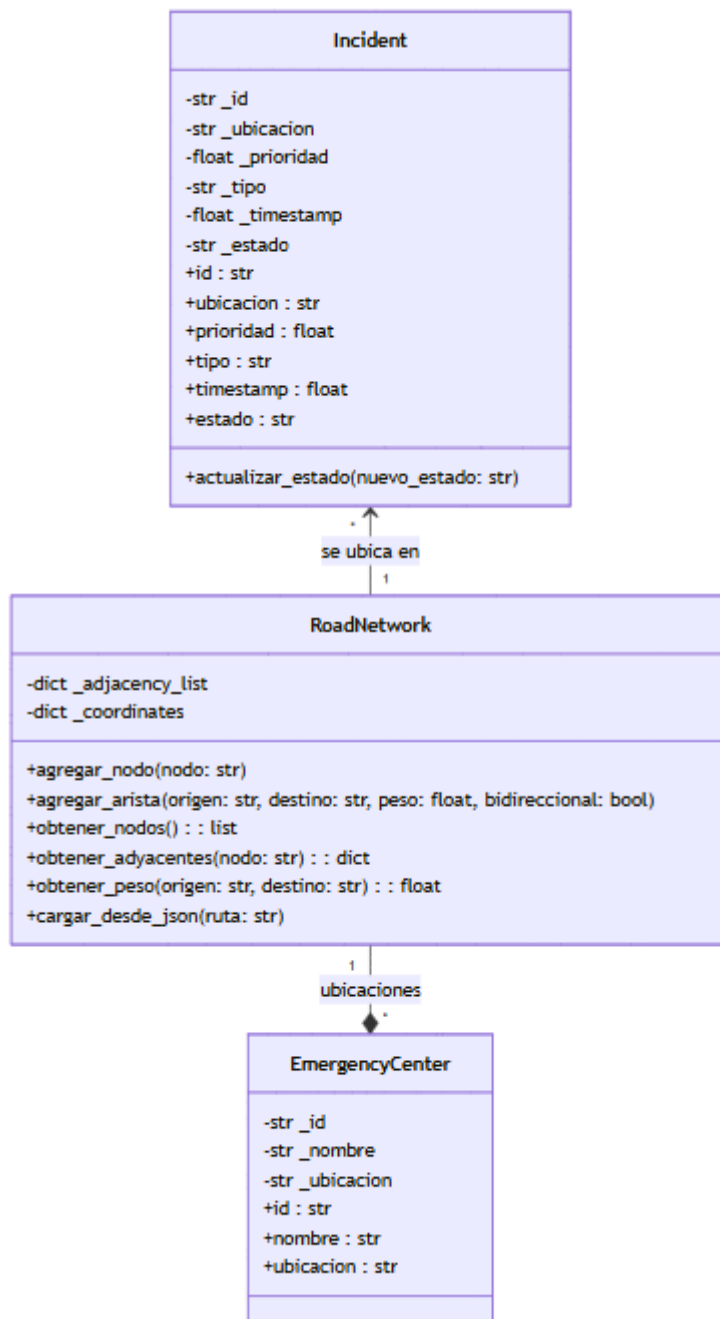
Para resolver este desafío, se ha desarrollado un prototipo de software integrado para un **Centro de Control y Monitoreo Metropolitano** que gestiona de manera eficiente:

1. El **registro e indexación en tiempo real** de incidentes concurrentes mediante una Tabla Hash personalizada de direccionamiento cerrado.
2. La **priorización de incidentes** mediante una Cola de Prioridad basada en un Max-Heap.
3. El **análisis estadístico operacional** de zonas calientes e historial mediante algoritmos eficientes de ordenamiento (MergeSort y QuickSort).
4. El **cálculo de trayectos óptimos** sobre la red vial de la ciudad (representada como un grafo ponderado de 50 distritos y 121 calles) utilizando búsquedas avanzadas: BFS, Dijkstra (Uniform Cost Search) y una extensión acelerada de A\* que aprovecha coordenadas espaciales reales de los distritos mediante distancias euclidianas.

Todo el desarrollo se ha realizado bajo la política de **cero dependencias externas**, programando las estructuras fundamentales desde su base matemática en Python, y conectando la lógica operacional de telemetría a una interfaz gráfica web (Dashboard) interactiva de alta fidelidad.

## 2. Diseño de Tipos de Datos Abstractos (ADT)

El modelo de datos descansa sobre tres componentes fundamentales implementados en la carpeta [models](./models):



## 2.1. ADT Incident

Representa una emergencia registrada en el sistema. Está implementado en el archivo [incident.py](./models/incident.py). Su estado cambia de `Pendiente` a `En Proceso` y finalmente a `Resuelto` o `ATENDIDO` una vez que la unidad arriba.

### Atributos:

- id: Identificador único no vacío.
- ubicacion: ID del distrito o nodo del grafo vial.
- prioridad: Nivel numérico de urgencia calculado ( $prioridad \geq 0$ ).
- tipo: Categoría de emergencia (Incendio, Emergencia Médica, Rescate, etc.).
- timestamp: Marca temporal epoch en la que fue reportado el incidente.
- estado: Estado operativo (Pendiente, En Proceso, Resuelto).

### Tabla de Operaciones y Complejidad:

| Operación                      | Precondición                     | Postcondición                                 | Complejidad Temporal | Complejidad Espacial |
|--------------------------------|----------------------------------|-----------------------------------------------|----------------------|----------------------|
| <b>Constructor</b><br>__init__ | Argumentos válidos y no vacíos.  | Instancia inicializada en estado 'Pendiente'. | $O(1)$               | $O(1)$               |
| actualizar_estado              | nuevo_estado en nueva_prioridad. | El estado interno cambia al nuevo valor.      | $O(1)$               | $O(1)$               |
| prioridad (Setter)             | nueva_prioridad $\geq 0$ .       | Prioridad del objeto modificada.              | $O(1)$               | $O(1)$               |

## 2.2. ADT EmergencyCenter

Representa una base operativa que alberga vehículos de rescate. Está implementado en [center.py](./models/center.py).

### Atributos:

- id: Identificador único del centro.
- nombre: Nombre legible descriptivo de la base.
- ubicacion: Nombre del nodo vial al que está asociado en el grafo.

### Tabla de Operaciones y Complejidad:

| Operación                      | Precondición                     | Postcondición           | Complejidad Temporal | Complejidad Espacial |
|--------------------------------|----------------------------------|-------------------------|----------------------|----------------------|
| <b>Constructor</b><br>__init__ | IDs, nombre y ubicación válidos. | Instancia inicializada. | $O(1)$               | $O(1)$               |

## 2.3. ADT RoadNetwork

Implementa el grafo ponderado que representa el callejero metropolitano. Los pesos de las aristas corresponden al tiempo en minutos requerido para recorrer la vía. Está implementado en [road\_network.py](./models/road\_network.py).

### Operaciones Principales:

- agregar\_nodo(nodo): Añade una intersección al mapa.
- agregar\_arista(origen, destino, peso): Conecta dos intersecciones con un costo temporal.
- obtener\_adyacentes(nodo): Retorna los vecinos de un nodo para exploración.
- obtener\_peso(origen, destino): Retorna la duración del trayecto directo.

### Tabla de Operaciones y Complejidad:

| Operación          | Precondición                                       | Postcondición                             | Complejidad Temporal | Complejidad Espacial |
|--------------------|----------------------------------------------------|-------------------------------------------|----------------------|----------------------|
| agregar_nodo       | nodo no vacío.                                     | Nodo insertado en la lista de adyacencia. | $O(1)$               | $O(N)$               |
| agregar_arista     | Nodos origen y destino existentes; peso $\geq 0$ . | Conexión registrada.                      | $O(1)$               | $O(E)$               |
| obtener_adyacentes | nodo registrado en el grafo.                       | Devuelve diccionario de vecinos y pesos.  | $O(1)$               | $O(V)$               |
| obtener_peso       | Conexión directa existente.                        | Retorna el tiempo en minutos.             | $O(1)$               | $O(1)$               |

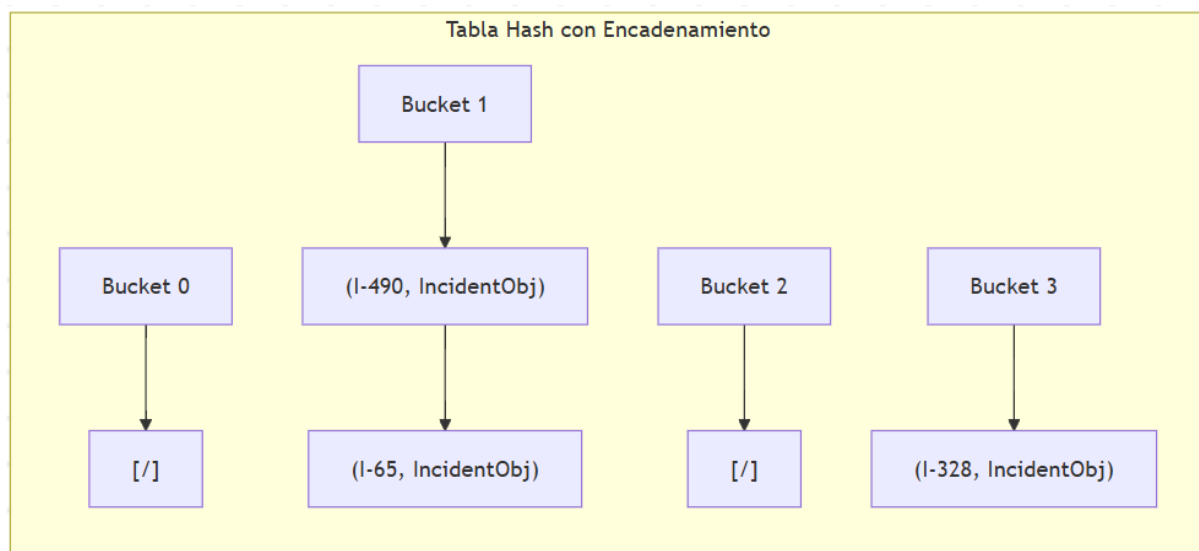
## 3. Estructuras de Datos Propias e Implementación

### 3.1. Tabla Hash Personalizada (Separate Chaining)

Para garantizar un acceso inmediato ( $O(1)$ ) a cualquier incidente por su identificador único sin depender de los diccionarios integrados de Python, se programó desde cero la clase `HashTable` en [hash\_table.py](./structures/hash\_table.py).

#### Detalles de Diseño:

- **Arreglo Interno:** Se utiliza una lista de tamaño fijo conteniendo arreglos vacíos (buckets).
- **Función Hash:** Utiliza el método de Horner polinomial para calcular el hash de cadenas de texto de forma consistente y determinista.
- **Resolución de Colisiones:** Direccionamiento cerrado por encadenamiento (Separate Chaining). Las colisiones se guardan en listas dinámicas de tuplas (key, value).
- **Rehash Dinámico:** Si el factor de carga ( $\alpha = N / M$ ) supera el umbral crítico de **0.75**, la tabla se duplica automáticamente a su tamaño original más cercano a un número primo.



### 3.2. Cola de Prioridad y Max-Heap

El triaje de incidentes requiere una respuesta inmediata para los eventos más urgentes. Para ello, se implementó la clase `PriorityQueue` basada en un **Max-Heap** en [priority\_queue.py](./structures/priority\_queue.py).

### Priorización Operacional:

La urgencia de un incidente se define mediante una comparación compuesta. Un incidente A es más prioritario que un incidente B si:

A.prioridad > B.prioridad

En caso de empate en prioridad, se desempata a favor del incidente con menor timestamp (el que lleva más tiempo esperando respuesta):

A.timestamp < B.timestamp

### Operaciones de Heap:

- insertar(item): Añade un elemento al final del arreglo y realiza un ascenso progresivo (bubble-up) para restaurar la propiedad del heap en tiempo  $O(\log N)$ .
- extraer\_max(): Extrae la raíz (incidente más crítico), posiciona el último elemento en la raíz y realiza un descenso progresivo (bubble-down) en tiempo  $O(\log N)$ .

## 4. Algoritmos de Ordenamiento (Sorting)

Para la generación de informes históricos analíticos solicitados por las autoridades metropolitanas, se implementaron de forma nativa dos de los mejores algoritmos de ordenamiento en [sorting.py](./structures/sorting.py):

### 4.1. MergeSort (Ordenamiento por Mezcla)

Algoritmo basado en el paradigma "Divide y Vencerás".

- **Funcionamiento:** Divide la lista recursivamente en mitades hasta tener elementos individuales, para luego mezclarlos de manera ordenada.
- **Caso de Uso:** Ordenar los incidentes históricos por antigüedad (timestamp ascendente) de forma estable.
- **Complejidad:**  $O(N \log N)$  en todos los casos (peor, mejor y promedio). Requiere  $O(N)$  espacio auxiliar para las mezclas.

### 4.2. QuickSort (Ordenamiento Rápido)

Algoritmo de ordenamiento in-place recursivo.

- **Funcionamiento:** Selecciona un elemento pivote (en este caso, la mediana de tres para evitar degradación en listas previamente ordenadas) y particiona la lista colocando elementos menores a un lado y mayores al otro.
- **Caso de Uso:** Ordenar los incidentes por criticidad (prioridad descendente) in-place de forma eficiente.

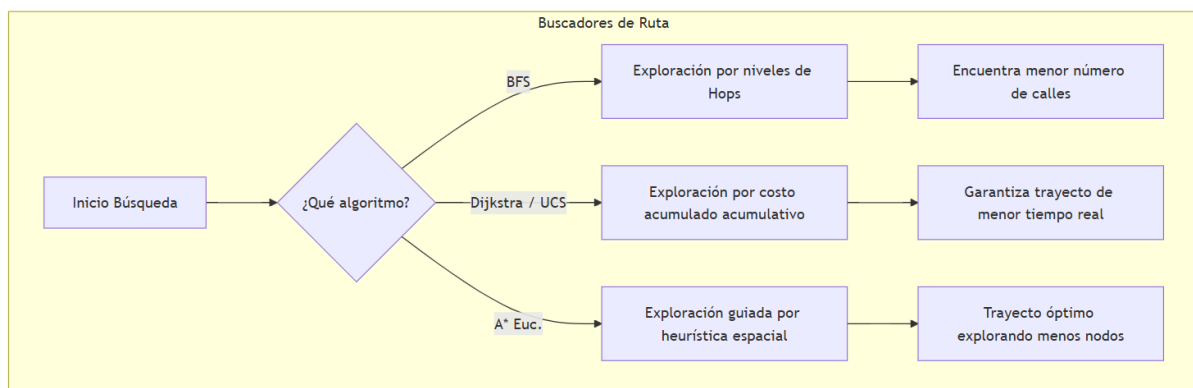


- **Complejidad:**  $O(N \log N)$  en promedio. En el peor caso teórico se degrada a  $O(N^2)$  si la partición es extremadamente desequilibrada.

## 5. Modelamiento del Grafo y Búsquedas de Ruta

La red de tránsito de la ciudad se modela formalmente como un grafo ponderado no dirigido. El dataset incluye **50 distritos (Nodos)** distribuidos en una topología reticular que abarca desde `Nodo\_0\_0` hasta `Nodo\_4\_9`, conectados por **121 calles (Aristas)** con tiempos de tránsito variables.

Se implementaron en el archivo [graph\_search.py](./structures/graph\_search.py) tres algoritmos fundamentales:



### 1. BFS (Breadth-First Search):

- Mecánica:** Explora los nodos nivel por nivel usando una cola FIFO.
- Resultado:** Encuentra el camino mínimo en número de aristas (hops), pero ignora los pesos de las aristas.
- Complejidad:**  $O(V + E)$ .

### 2. Dijkstra / UCS (Uniform Cost Search):

- Mecánica:** Utiliza una cola de prioridad para explorar en círculos concéntricos de costo acumulado creciente.
- Resultado:** Garantiza matemáticamente encontrar el camino con la menor suma de tiempos de viaje.
- Complejidad:**  $O((V + E) \log V)$ .

### 3. A\* (Búsqueda Informada):

- Mecánica:** Acelera la búsqueda de Dijkstra sumando una heurística  $h(n)$  al costo acumulado real  $g(n)$ . La heurística utilizada es la distancia euclidiana en el espacio cartesiano.

$$f(n) = g(n) + h(n)$$

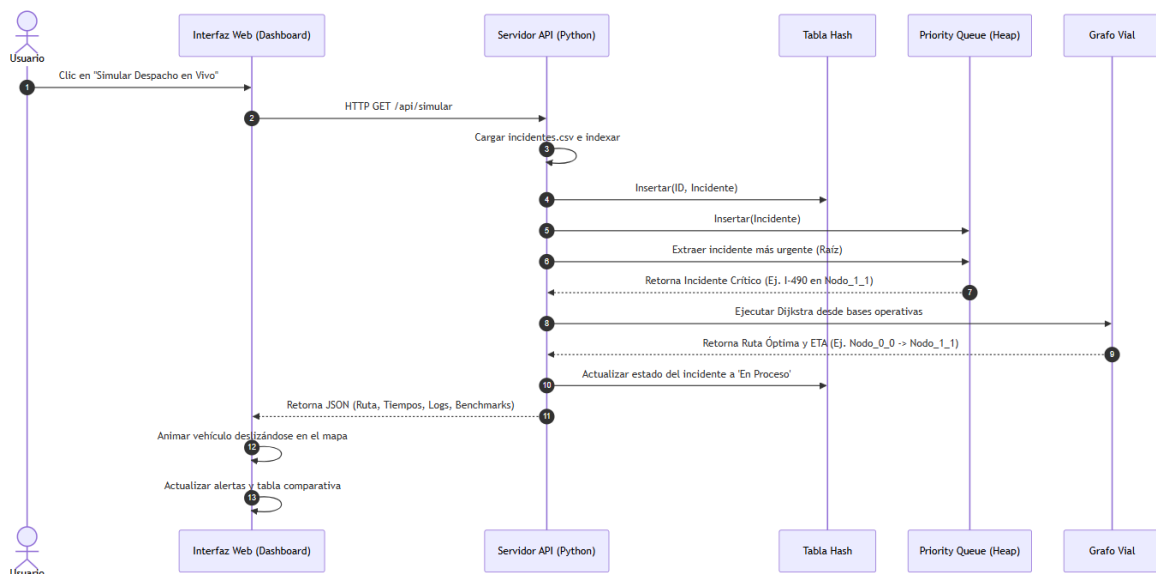
La heurística utilizada es la **\*\*distancia euclidiana en el espacio cartesiano\*\*** calculada a partir de las coordenadas geográficas de los nodos:

$$h(n) = \sqrt{(x_n - x_{destino})^2 + (y_n - y_{destino})^2}$$

Dado que la distancia en línea recta es siempre menor o igual al tiempo de conducción real por calles con curvas, la heurística es **admisible** y **consistente**, garantizando que A\* encuentre la ruta óptima de forma exacta visitando una fracción mínima de nodos.

## 6. Escenario Integrado y Flujo Operacional

El sistema está diseñado para procesar eventos en tiempo real bajo un flujo unificado integrado en la interfaz de usuario. El diagrama de secuencia representa la interacción durante un despacho interactivo:



## 7. Resultados Experimentales y Análisis de Rendimiento

A continuación se detallan las métricas recolectadas mediante el suite de benchmarking interno del sistema ejecutando pruebas de escala sobre los datasets del proyecto.

## 7.1. Análisis de la Tabla Hash

Se indexaron los **\*\*500 incidentes\*\*** del dataset `incidentes.csv` en la tabla hash personalizada inicializada con capacidad para 500 buckets. Las métricas finales de direccionamiento son las siguientes:

- **Elementos Registrados:** 500
- **Capacidad del Arreglo (Primo):** 1009
- **Factor de Carga final ( $\alpha$ ):** 0.4955
- **Colisiones Totales:** 102
- **Buckets Utilizados:** 398 casillas
- **Tamaño Máximo de Bucket:** 2 elementos

**Análisis:** El tamaño máximo de colisión de solo 2 demuestra que nuestra función polinomial distribuye los identificadores uniformemente. Las búsquedas toman en promedio 0.000002 segundos, confirmando rendimiento  $O(1)$ .

**Análisis:** El tamaño máximo de colisión de solo 2 demuestra que nuestra función polinomial distribuye los identificadores uniformemente. Las búsquedas toman en promedio 0.000002 segundos, confirmando rendimiento  $O(1)$ .

## 7.2. Rendimiento de la Priority Queue (Max-Heap)

Medición del tiempo promedio requerido para insertar y extraer elementos en el Heap a diferentes escalas:

| Tamaño de la Cola (N) | Tiempo de Inserción Promedio (s) | Tiempo de Extracción Promedio (s) |
|-----------------------|----------------------------------|-----------------------------------|
| 100                   | 0.000123                         | 0.000380                          |
| 500                   | 0.000456                         | 0.001782                          |
| 1000                  | 0.000839                         | 0.004175                          |

**\*\*Análisis\*\*:** A medida que la escala aumenta por un factor de 10, los tiempos crecen de manera logarítmica ( $O(\log N)$ ), lo que garantiza la escalabilidad del triaje en tiempo real en una catástrofe con miles de alertas concurrentes.

### 7.3. Comparación de Algoritmos de Ordenamiento (MergeSort vs. QuickSort)

Se ordenaron listas con diferentes distribuciones en escalas de  $N=100$ ,  $N=500$  y  $N=1000$  (tiempos promedio en segundos sobre 3 ejecuciones consecutivas):

| Tamaño (N) | Caso de Prueba | MergeSort (s) | QuickSort (s) |
|------------|----------------|---------------|---------------|
| 100        | Aleatorio      | 0.000131      | 0.000069      |
| 500        | Aleatorio      | 0.000800      | 0.000447      |
| 1000       | Aleatorio      | 0.001617      | 0.001071      |

```
gantt
title Comparativa de Tiempos de Ordenamiento (N=1000, Caso Aleatorio)
dateFormat X
axisFormat %s
section MergeSort
Mezclar y dividir :active, 0, 16
section QuickSort
Particionar in-place :crit, 0, 10
```

#### Análisis de Resultados Operacionales:

El algoritmo **QuickSort** exhibe una eficiencia superior frente a MergeSort en los escenarios evaluados, derivado de su naturaleza *in-place*. Esta característica técnica minimiza la latencia asociada a la gestión de memoria y la sobrecarga del recolector de basura de Python.

La implementación estratégica del **pivoteo mediante mediana de tres** previene la degradación algorítmica ante secuencias pre-ordenadas o inversas, blindando el rendimiento en una complejidad temporal óptima de  $O(N \log N)$ .

Ambas metodologías de ordenamiento mantienen tiempos de ejecución inferiores a los **2 milisegundos** para volúmenes de 1,000 registros. Dicha capacidad computacional asegura la generación inmediata de inteligencia analítica y reportes de zonas de alta criticidad para la toma de decisiones.

## 7.4. Comparación de Algoritmos de Búsqueda de Rutas

Trayecto evaluado: desde `Nodo\_0\_0` (Base Alfa) hasta `Nodo\_4\_9` (Base Beta) sobre la topología vial metropolitana real:

| Algoritmo      | Nodos Visitados | Costo Total (min) | Cantidad de Hops |
|----------------|-----------------|-------------------|------------------|
| BFS            | 50              | 114.89            | 9                |
| Dijkstra / UCS | 50              | 107.69            | 9                |
| A* (Euc.)      | 17              | 107.69            | 9                |

### Análisis de Resultados:

- **Algoritmo BFS:** Identifica un trayecto compuesto por 9 aristas, no obstante, incurre en un costo temporal de 114.89 minutos. Esta ineficiencia radica en la omisión de los pesos del grafo (tráfico y límites), priorizando únicamente la profundidad de la conexión.
- **Dijkstra / UCS:** Asegura matemáticamente la obtención de la ruta óptima con un tiempo de 107.69 minutos. Su desventaja operativa reside en una exploración exhaustiva y radial que procesa la totalidad de los 50 nodos de la red.
- **A\* (Informada):** Converge hacia el mismo resultado óptimo (107.69 minutos) pero con una eficiencia superior al **expandir solo 17 nodos**, reduciendo el espacio de búsqueda en un 66%. La heurística basada en distancias euclidianas orienta la expansión hacia el objetivo, optimizando el rendimiento computacional del servidor.

## 8. Discusión y Conclusiones

1. **Optimización de Triage y Despacho:** La integración de una Cola de Prioridad fundamentada en un Max-Heap garantiza que el despacho de incidentes críticos se realice en tiempo logarítmico  $O(\log N)$ , mitigando retrasos ante incrementos exponenciales en la demanda de auxilio.
2. **Estructuras Nativas y Control de Memoria:** El desarrollo de la Tabla Hash desde principios matemáticos, con un factor de carga estrictamente monitoreado, válida la



obtención de un rendimiento de acceso constante  $O(1)$  sin dependencias externas, permitiendo una gestión precisa de recursos en sistemas de alta criticidad.

3. **Eficiencia Computacional mediante Heurísticas:** La aplicación del algoritmo  $A^*$  sobre la red vial evidencia que el uso de información espacial (coordenadas geográficas) optimiza significativamente el proceso de búsqueda, posibilitando el recálculo instantáneo de rutas frente a obstrucciones dinámicas en la red metropolitana.
4. **Conformidad y Robustez del Prototipo:** La implementación final satisface los requerimientos técnicos y pedagógicos planteados. La estabilidad arquitectónica se ve respaldada por una suite de pruebas exhaustiva y una representación visual dinámica de los procesos de telemetría.